

Coding Conventions

이 문서는 Python 코드를 일관되고 읽기 쉽게 작성하기 위한 기본 규칙을 정의한다. 모든 신규 코드와 수정 코드는 아래 기준을 따른다.

1. 이름 짓기

이름만 보고 역할을 알 수 있도록 의미가 분명한 단어를 사용한다.

대상	작성 방식	기준	예시
클래스	PascalCase	각 단어의 첫 글자를 대문자로 작성한다.	VideoStreamManager, BufferInfo
함수	snake_case	모든 단어를 소문자로 작성하고 단어 사이에 밑줄을 넣는다. 일반적으로 동사로 시작한다.	add_buffer(), initialize_stream()
변수	snake_case	모든 단어를 소문자로 작성하고 단어 사이에 밑줄을 넣는다.	buffer_size, input_path

약어나 지나치게 짧은 이름보다 목적이 드러나는 이름을 우선한다.

```
# 올바른 예
buffer_size = 1024
input_path = "data/input.json"

# 피해야 할 예: 역할을 알기 어렵다.
bs = 1024
p = "data/input.json"
```

2. 공백과 연산자

대입, 산술, 비교 등 이항 연산자의 앞뒤에는 공백을 한 칸씩 넣는다.

```
# 올바른 예
number = minor + major
is_valid = count < limit

# 피해야 할 예
number=minor+major
```

3. 들여쓰기와 코드 블록

- 들여쓰기는 **공백 4칸**을 권장한다.
- 탭을 사용할 수 있지만, 하나의 파일 안에서 탭과 공백을 섞지 않는다.
- 함수, 클래스와 **if**, **for**, **while** 등의 코드 블록마다 한 단계씩 들여쓴다.

- 블록을 시작하는 문장 끝에는 콜론(:)을 붙인다.
- 같은 블록에 속한 코드는 동일한 깊이로 들여쓴다.

```
while count < 5:
    print(f"Hello world! {count}")
    count += 1
```

4. 빈 줄

논리적으로 하나의 작업이 끝나고 다음 작업이 시작될 때 빈 줄을 넣는다. 연관된 문장 사이에는 불필요한 빈 줄을 넣지 않는다.

- 최상위 함수와 클래스 사이는 두 줄 띄운다.
- 클래스 내부의 메서드 사이는 한 줄 띄운다.
- 함수 내부에서는 작업 단위를 구분할 때 한 줄 띄운다.

```
def print_iterations(limit):
    count = 0

    while count < limit:
        print(f"Hello world! {count}")
        count += 1

    return count

def main():
    limit = int(input("반복 횟수: "))
    print_iterations(limit)
```

5. 주석

주석은 코드가 무엇을 하는지 또는 왜 필요한지 설명할 때 사용한다. Python에서는 함수 설명에 docstring을 사용하고, 코드 블록과 한 문장에 대한 설명에는 # 주석을 사용한다.

함수 주석

함수 선언 바로 아래에 큰따옴표 3개('''')로 감싼 docstring을 작성한다. 함수의 이름, 기능, 인자, 반환값, 작성 날짜를 설명한다.

```
def add_two_integers(operand1, operand2):
    """
    함수 이름: add_two_integers()
    기능: 두 인자를 받아 더한 값을 반환한다.
    인자: operand1 -> 첫번째 숫자
         operand2 -> 두번째 숫자
    반환값: 두 숫자의 합
    """
```

```

    작성 날짜: 2026/08/10
    """
    return operand1 + operand2

```

클래스 주석

클래스 선언 바로 아래에 큰따옴표 3개(""")로 감싼 docstring을 작성한다. 클래스의 이름, 기능, 작성 날짜를 설명한다.

```

class AccountInfo:
    """
    클래스 이름: AccountInfo
    기능: 계좌의 잔액 정보를 관리한다.
    작성 날짜: 2026/08/12
    """

    def __init__(self, asset_name, available_balance):
        self.asset_name = asset_name
        self.available_balance = available_balance

```

블록 주석

여러 문장이 하나의 작업을 수행할 때 해당 코드 바로 위에 # 주석을 작성한다.

```

# 사용자에게 두 정수를 입력받아 더한 뒤 total_value에 저장한다.
input1 = int(input("첫 번째 정수: "))
input2 = int(input("두 번째 정수: "))
total_value = input1 + input2

```

문장 주석

한 문장만 설명할 때는 코드 뒤에 두 칸 이상 띄우고 # 주석을 작성한다.

```

print(f"총합: {total_value}") # 총합을 화면에 출력한다.

```

6. 작성 전 확인 목록

- 이름만 보고 클래스, 함수, 변수의 역할을 알 수 있는가?
- 클래스는 `PascalCase`, 함수와 변수는 `snake_case`인가?
- 이항 연산자 앞뒤에 공백이 있는가?
- 들여쓰기 방식이 파일 전체에서 일관적인가?
- 코드 블록을 시작하는 문장 끝에 콜론이 있는가?
- 작업 단위 사이에만 필요한 빈 줄을 넣었는가?
- 함수 설명은 docstring으로, 코드 설명은 # 주석으로 작성했는가?